Trabajo en Clase 03 — Docker y Docker Compose

Integrantes

Jhonnatan David Hernandez Martinez - 2201492
Diego Alejandro Gómez Pedraza - 2211060
Luis Sebastián Mora Cañas - 2211554

Se contenerizó la aplicación DemoAcademico y se ejecutó junto con MySQL usando Docker Compose.

Actividades

1. Se verificó la instalación de Docker con `docker --version` y `docker info`.
2. Se ejecutó el contenedor de prueba `hello-world`.
3. Se levantaron dos servidores nginx en los puertos `8088` y `8089` del host (ambos usan el 80 interno).
4. Se compiló la aplicación con `mvn clean package`.
5. Se corrigió un error de arranque del JAR: `javafaker` traía una versión vieja de SnakeYAML que chocaba con Spring Boot; se excluyó en el `pom.xml`.
6. Se creó el `Dockerfile` en la raíz del proyecto. Se agregó `ENV SERVER_PORT=8080` porque el `application.yaml` usa el puerto 8081.
7. Se construyó la imagen `demoacademico` y se ejecutó el contenedor en el puerto `8080`.
8. Se creó el `docker-compose.yml` con los servicios `app` y `db` (MySQL 8.0) y el volumen `mysql_data`.
9. Se ejecutó `docker compose up -d`, se revisaron servicios, logs y volúmenes, y se detuvo con `docker compose down`.

Captura 1 — Contenedor de prueba (hello-world)

```
$ docker run hello-world
…
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

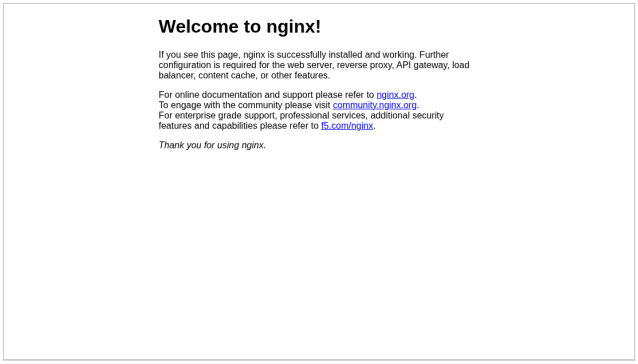
To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Captura 2 — Servidor nginx (localhost:8088 y localhost:8089)

```
$ docker run -d -p 8088:80 --name nginx1 nginx && docker run -d -p 8089:80 --name nginx2 nginx && docker
ps
a245f043b400a4fc3bfbf5119c544e9de415cf2f8f8b4f5c8ba264a4bc104172
7394013bd03e4610f49ba9834568efb529e8e3755f1c16b09c972ae3406b6162
CONTAINER ID   IMAGE     COMMAND                  CREATED                  STATUS                  PORTS
NAMES
7394013bd03e   nginx     "/docker-entrypoint.…"   Less than a second ago   Up Less than a second   0.0.0.
0:8089->80/tcp, [::]:8089->80/tcp   nginx2
a245f043b400   nginx     "/docker-entrypoint.…"   Less than a second ago   Up Less than a second   0.0.0.
0:8088->80/tcp, [::]:8088->80/tcp   nginx1
```





Captura 3 — Construcción de imagen (docker build y docker images)

```
$ docker build -t demoacademico . && docker images demoacademico
…
 ---> Using cache
 ---> 5c7e342ca77a
Step 8/9 : EXPOSE 8080
 ---> Using cache
 ---> 3f17f0bd0e0b
Step 9/9 : ENTRYPOINT ["java","-Xmx1024m","-jar","app.jar"]
 ---> Using cache
 ---> d68b6492fa7c
Successfully built d68b6492fa7c
Successfully tagged demoacademico:latest
REPOSITORY      TAG       IMAGE ID       CREATED        SIZE
demoacademico   latest    d68b6492fa7c   14 hours ago   531MB
```

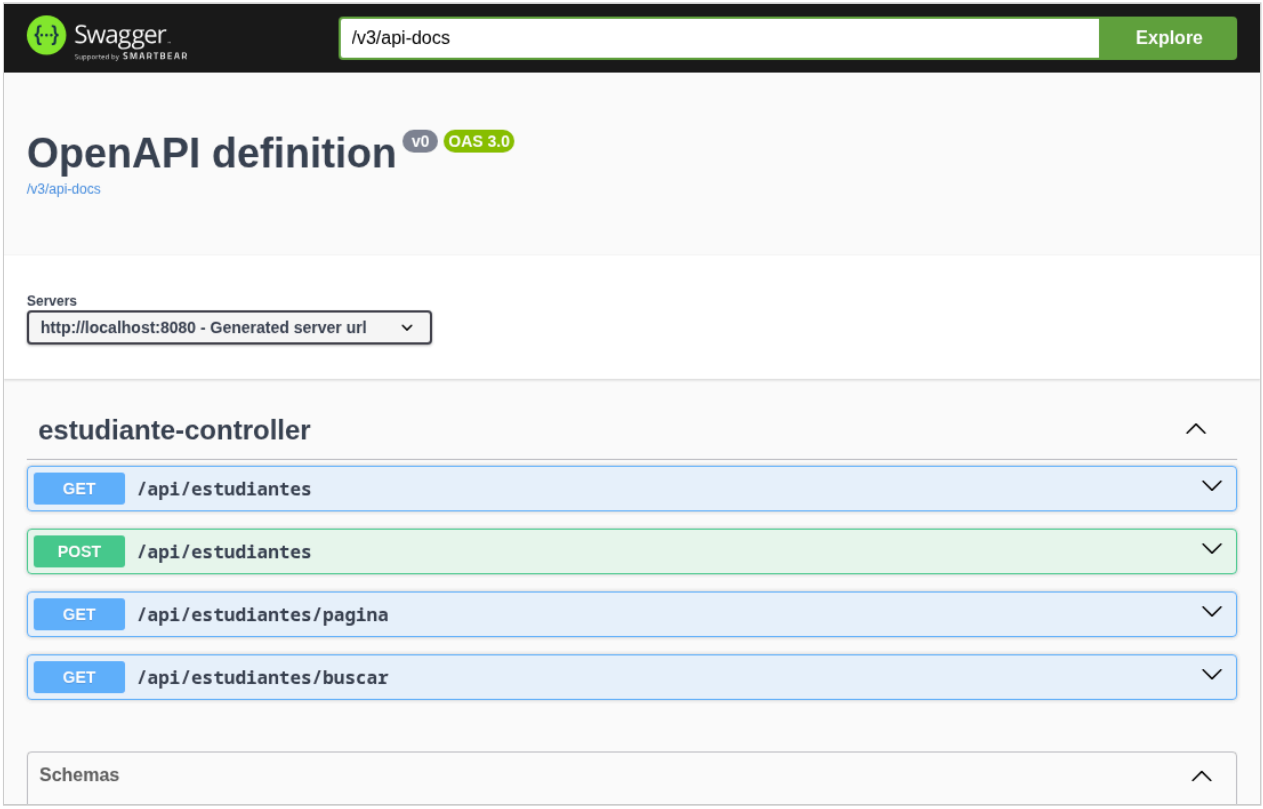
Captura 4 — Contenedor de la aplicación (docker ps y prueba del servicio)

```
$ docker ps --filter name=demoacademico
CONTAINER ID   IMAGE           COMMAND                  CREATED         STATUS         PORTS
NAMES
50ce3bf89d3f   demoacademico   "java -Xmx1024m -jar…"   9 seconds ago   Up 9 seconds   0.0.0.0:8080->808
0/tcp, [::]:8080->8080/tcp   demoacademico

$ curl -s 'http://localhost:8080/api/estudiantes/pagina?page=0&size=3' | head -c 600; echo
{"content":[{"id":1,"nombre":"Sta. Juan Mireles Granado","email":"est0_gerardo.guillen@gmail.com"},{"id":
2,"nombre":"Sra. Débora Quintana Patiño","email":"est1_lucas.sanabria@yahoo.com"},{"id":3,"nombre":"Olivi
a Meza Granado","email":"est2_analuisa.carrion@hotmail.com"}],"pageable":{"pageNumber":0,"pageSize":3,"so
rt":{"sorted":false,"empty":true,"unsorted":true},"offset":0,"paged":true,"unpaged":false},"last":fals
e,"totalPages":16,"totalElements":47,"first":true,"size":3,"number":0,"sort":{"sorted":false,"empty":tru
e,"unsorted":true},"numberOfElements":3,"empty":false}
```

Captura 5 — Docker Compose (aplicación y MySQL)

```
$ docker compose ps
NAME                IMAGE           COMMAND                  SERVICE   CREATED          STATUS         PO
RTS
demoacademico-app   demoacademico   "java -Xmx1024m -jar…"   app       10 seconds ago   Up 9 seconds   0.
0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
demoacademico-db    mysql:8.0       "docker-entrypoint.s…"   db        10 seconds ago   Up 9 seconds   33
060/tcp, 0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
```



Captura 6 — Volúmenes

```
$ docker volume ls
DRIVER    VOLUME NAME
local     3-material-actividades_docker-cache
local     3-material-actividades_jenkins-data
local     3-material-actividades_jenkins-docker-certs
local     84cb42c158133f4f859cf7b8ffeee7b55181a38201390c868b0e773542357a5f
local     815d714297b4f629969aa97fec9b9a6b11d6827656a8ab709f9ac8676b663e91
local     a4ebc23032724e176db5f76e44aa942649c8aea394290c22a7800bed183a94d6
local     ab7fb8d8c7e71d213208fa00ace3697b03dd064856c9f34504093d3c09cadcae
local     bc9f94745368f22b23ac62a043d6f847c2a15530a96e288627e61e1ec18e1193
local     cd1fc033495573f4690b253c7a58d319f392dc08832ca52f2f656ea586790c5d
local     de94fa107b9e63d7d6efbe34c938b52912fdbe4f541d913eafb8581d06577d91
local     demoacademicoapplication_mysql_data
local     juego_software_postgres_data
local     odysseus_chromadb-data
local     odysseus_ntfy-cache
local     odysseus_searxng-data

$ docker compose down && docker volume ls | grep mysql_data
…
local     demoacademicoapplication_mysql_data
```

El volumen `demoacademicoapplication_mysql_data` se mantiene después de `docker compose down`.

Reflexión técnica

1. **¿Cuál es la diferencia entre imagen y contenedor?** La imagen es la plantilla con el sistema base, las dependencias y la aplicación. El contenedor es una instancia en ejecución de esa imagen; de una misma imagen salen varios contenedores, como `nginx1` y `nginx2`.
2. **¿Qué problema de ingeniería de software ayuda a resolver Docker?** El de “en mi computador funciona”. La aplicación se empaqueta con su entorno (Java 21, librerías, configuración), así que corre igual en cualquier máquina.
3. **¿Qué ventaja ofrece Docker Compose frente a ejecutar contenedores manualmente?** Toda la configuración (servicios, puertos, variables, volúmenes y dependencias) queda en un solo archivo YAML, y la solución completa se levanta con `docker compose up` en lugar de varios `docker run`.
4. **¿Por qué es importante usar volúmenes con bases de datos?** Porque los datos guardados dentro del contenedor se pierden al eliminarlo. El volumen `mysql_data` guarda `/var/lib/mysql` fuera del contenedor y conserva la información.
5. **¿Qué significa el mapeo `8080:8080` y `3307:3306`?** El formato es `puertoHost:puertoContenedor`. El 8080 del computador lleva al 8080 de la aplicación, y el 3307 del computador lleva al 3306 de MySQL (se usa 3307 para no chocar con un MySQL local).